



CETAM
CENTRO DE EDUCAÇÃO
TECNOLOGICA DO AMAZONAS



GOVERNO DO
AMAZONAS

Qualificação Profissional em Lógica de Programação Básica

APOSTILA DE

LÓGICA DE PROGRAMAÇÃO BÁSICA

com Portugol e VisuAlg

Material didático para alunos iniciantes em programação

Professora: Esp. Priscila Gonçalves

E-mail leylapristeacher@gmail.com

GitHb: https://github.com/priscilagon/Logica_Programacao_Basica

Manaus – Amazonas

Sumário

Capítulo 1: Introdução à Lógica de Programação	5
1.1 O que é lógica de programação?	5
1.2 Raciocínio lógico no dia a dia	5
1.3 Abstração e resolução de problemas.....	5
1.4 Exercícios: Capítulo 1	6
Capítulo 2: Algoritmos e Fluxogramas.....	7
2.1 O que é um algoritmo?.....	7
Características de um bom algoritmo.....	7
2.2 Formas de representar um algoritmo.....	7
2.3 Símbolos básicos de fluxograma.....	7
2.4 Exemplo do problema ao algoritmo.....	9
2.5 Exercícios: Capítulo 2	9
Capítulo 3: Conhecendo o VisuAlg e o Portugol	10
3.1 O que é o Portugol?	10
3.2 O que é o VisuAlg?	10
3.3 Estrutura básica de um algoritmo em VisuAlg	10
Explicando cada parte.....	10
3.4 Primeiro programa: "Olá, Mundo!"	10
3.5 Regras gerais de sintaxe.....	11
3.6 Exercícios: Capítulo 3	11
Capítulo 4: Variáveis, Constantes e Tipos de Dados.....	12
4.1 O que são variáveis?.....	12
4.2 Regras para nomear variáveis.....	12
4.3 Tipos de dados primitivos no VisuAlg.....	12
4.4 Declarando variáveis	12
4.5 Constantes	12
4.6 Atribuição de valores.....	13
4.7 Exercícios: Capítulo 4	13
Capítulo 5: Operadores e Expressões	14
5.1 Operadores aritméticos.....	14
5.2 Operadores relacionais.....	14
5.3 Operadores lógicos	14
5.4 Ordem de precedência dos operadores.....	15
5.5 Exemplo prático.....	15

5.6 Exercícios: Capítulo 5	15
Capítulo 6: Entrada e Saída de Dados.....	16
6.1 Comandos de saída: escreva() e escreval()	16
6.2 Comando de entrada: leia()	16
6.3 Concatenando textos e variáveis	16
6.4 Exemplo completo: Calculadora de Média	17
6.5 Exercícios: Capítulo 6	17
Capítulo 7: Estruturas Condicionais	18
7.1 Por que usar estruturas condicionais?	18
7.2 Seleção simples: se ... entao	18
7.3 Seleção composta: se ... entao ... senao	18
7.4 Condições encadeadas: senao se	19
7.5 Seleção múltipla: escolha ... caso	19
7.6 Exercícios: Capítulo 7	20
Capítulo 8: Estruturas de Repetição.....	21
8.1 Por que usar estruturas de repetição?	21
8.2 Estrutura para (contador definido)	21
8.3 Estrutura enquanto (repetição com teste no início)	21
8.4 Estrutura repita ... ate (repetição com teste no final).....	22
8.5 Comparando as três estruturas	22
8.6 Exercícios: Capítulo 8	23
Capítulo 9: Vetores (Introdução).....	24
9.1 O que é um vetor?	24
9.2 Declarando um vetor	24
9.3 Acessando e preenchendo um vetor	24
9.4 Busca em um vetor	25
9.5 Exercícios: Capítulo 9	25
Capítulo 10: Matrizes.....	26
10.1 O que é uma matriz?	26
10.2 Declarando uma matriz	26
10.3 Acessando elementos de uma matriz	26
10.4 Preenchendo e exibindo uma matriz.....	26
10.5 Exemplo: soma dos elementos de uma matriz	27
10.6 Exercícios: Capítulo 10	27
Capítulo 11: Arquivos	28
11.1 Por que utilizar arquivos?	28
11.2 Tipos de arquivo no VisuAlg	28
11.3 Comandos básicos para manipular arquivos.....	28
11.4 Gravando dados em um arquivo.....	28
11.5 Lendo dados de um arquivo	29
11.6 Exercícios: Capítulo 11	29
Capítulo 12: Projeto Prático Final.....	31

12.1 Objetivo do projeto.....	31
12.2 Proposta: Sistema de Cadastro de Produtos	31
12.3 Estrutura sugerida do algoritmo.....	31
12.4 Desafios extras	33
Referências	34

Capítulo 1: Introdução à Lógica de Programação

1.1 O que é lógica de programação?

Lógica de programação é a forma organizada de pensar que usamos para resolver problemas por meio de uma sequência de passos claros e ordenados. Antes de escrever qualquer linha de código em uma linguagem de programação, é preciso desenvolver o raciocínio lógico: a capacidade de analisar um problema, dividi-lo em partes menores e definir, passo a passo, como resolvê-lo.

Programar não é decorar comandos de uma linguagem. Programar é, antes de tudo, pensar de forma estruturada. Por isso, este material tem como ponto de partida o desenvolvimento do raciocínio lógico, para só depois apresentar a sintaxe da linguagem Portugol, utilizada no ambiente VisuAlg.

1.2 Raciocínio lógico no dia a dia

Sem perceber, usamos lógica em diversas situações cotidianas. Preparar uma receita, trocar um pneu, montar um móvel ou até escolher o caminho mais curto para o trabalho são tarefas que seguem uma sequência lógica de passos. Observe o exemplo a seguir:

Tarefa: Fazer um café
1. Colocar água na chaleira
2. Ligar a chaleira e esperar ferver
3. Colocar pó de café no filtro
4. Despejar a água quente sobre o pó
5. Servir o café na xícara

Perceba que essa sequência precisa seguir uma ordem: não é possível servir o café antes de coar, nem coar antes de ferver a água. Essa mesma lógica de sequência ordenada de passos é a base de qualquer algoritmo de programação.

1.3 Abstração e resolução de problemas

Abstração é a habilidade de identificar, em um problema real, apenas as informações essenciais para resolvê-lo, deixando de lado detalhes que não são relevantes para a solução. Ao programar, precisamos abstrair o problema do mundo real e traduzi-lo em dados, decisões e ações que o computador consiga executar.

Para desenvolver o raciocínio lógico, é útil seguir um método simples ao resolver qualquer problema:

1. Entender o problema: o que exatamente precisa ser resolvido?
2. Identificar os dados de entrada: quais informações eu tenho ou preciso pedir ao usuário?
3. Definir o processamento: quais cálculos, comparações ou decisões são necessários?
4. Definir a saída: qual é o resultado final que deve ser apresentado?
5. Testar a solução: a lógica funciona para diferentes situações e casos?

Dica da professora

Sempre que for resolver um exercício de lógica, escreva em português comum (sem se preocupar com comandos) o passo a passo da solução antes de tentar programar. Essa técnica é chamada de pseudocódigo livre e ajuda muito a organizar as ideias.

1.4 Exercícios: Capítulo 1

1. Descreva, em uma sequência numerada de passos, como você faria para trocar uma lâmpada queimada.
2. Escreva o passo a passo lógico para atravessar uma rua com segurança.
3. Pense em uma tarefa do seu cotidiano e identifique: quais são os dados de entrada, o processamento e a saída dessa tarefa?
4. Explique, com suas palavras, a diferença entre lógica de programação e uma linguagem de programação.

Capítulo 2: Algoritmos e Fluxogramas

2.1 O que é um algoritmo?

Um algoritmo é uma sequência finita, ordenada e não ambígua de passos que leva à solução de um problema. Todo algoritmo precisa ter um início, um conjunto de instruções claras e um fim. As instruções devem ser precisas o suficiente para que qualquer pessoa (ou computador) consiga executá-las sem margem para interpretações diferentes.

Características de um bom algoritmo

- **Finitude:** deve ter um número limitado de passos e terminar em algum momento.
- **Clareza:** cada passo deve ser bem definido, sem ambiguidade.
- **Precisão na ordem:** a sequência dos passos deve ser respeitada.
- **Eficácia:** deve realmente resolver o problema proposto.

2.2 Formas de representar um algoritmo

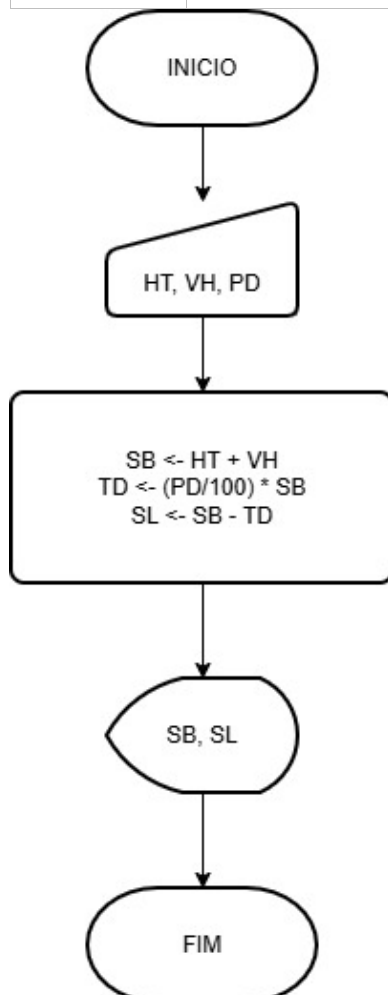
Existem três formas mais comuns de representar um algoritmo antes (ou no lugar) de escrevê-lo em uma linguagem de programação:

- **Descrição narrativa:** o algoritmo escrito em linguagem natural, como fizemos no Capítulo 1.
- **Fluxograma:** representação gráfica utilizando símbolos padronizados (retângulos, losangos, setas etc.).
- **Pseudocódigo (Portugol):** uma linguagem intermediária entre o português e uma linguagem de programação real, com uma sintaxe estruturada. É o que utilizaremos no VisuAlg.

2.3 Símbolos básicos de fluxograma

Símbolo	Nome	Significado
	Terminal <i>Terminator</i>	O símbolo representa a definição de início e fim do fluxo lógico de um programa. Também é utilizado na definição de sub-rotinas de procedimento ou de função.
	Entrada Manual <i>Manual input</i>	Representa a entrada manual de dados, normalmente efetuada em um teclado conectado diretamente ao console do computador.
	Processamento <i>Process</i>	Representa a execução de uma operação ou grupo de operações que estabelecem o resultado de uma operação lógica ou matemática.
	Exibição <i>Display</i>	Representa a execução da operação de saída visual de dados em um monitor de vídeo conectado ao console do computador.
	Decisão <i>Decision</i>	O símbolo representa o uso de desvios condicionais para outros pontos de programa de acordo com situações variáveis.

Símbolo	Nome	Significado
	Preparação <i>Preparation</i>	Representa a modificação de instruções ou grupo de instruções existentes em relação à ação de sua atividade subsequencial.
	Processo predefinido <i>Predefined process</i>	Definição de um grupo de operações estabelecidas como uma sub-rotina de processamento anexa ao diagrama de blocos.
	Conector <i>Connector</i>	Representa a entrada ou a saída em outra parte do diagrama de blocos. Pode ser usado da definição de quebras de linhas e na continuação da execução de decisões.
	Linha <i>Line</i>	O símbolo representa a ação de vínculo existente entre os vários símbolos de um diagrama de blocos. Possui a ponta de uma seta indicando a direção do fluxo de ação.



```

1 Algoritmo "CalcularSalarioProfessor"
2 // Data atual : 04/09/2026
3 Var
4 // Seção de Declarações das variáveis
5 HT : Real // Horas trabalhadas no mês
6 VH : Real // Valor hora-aula
7 PD : Real // Percentual de desconto
8 SB : Real // Salário bruto
9 TD : Real // Total de desconto
10 SL : Real // Salário líquido
11
12 Inicio
13 // Seção de Comandos, procedimento, funções, operadores, etc...
14 // 1. Estabelecer a leitura da variável HT
15 Escreva("Digite a quantidade de horas trabalhadas no mês (HT): ")
16 Leia(HT)
17
18 // 2. Estabelecer a leitura da variável VH
19 Escreva("Digite o valor da hora-aula (VH): ")
20 Leia(VH)
21
22 // 3. Estabelecer a leitura da variável PD
23 Escreva("Digite o percentual de desconto do INSS (PD): ")
24 Leia(PD)
25
26 // 4. Calcular o salário bruto (SB)
27 SB <- HT * VH
28
29 // 5. Calcular o total de desconto (TD)
30 TD <- (PD / 100) * SB
31
32 // 6. Calcular o salário líquido (SL)
33 SL <- SB - TD
34
35 // 7. Apresentar os valores dos salários bruto e líquido
36 Escreva("Salário Bruto (SB): R$ ", SB:1:2)
37 Escreva("Salário Líquido (SL): R$ ", SL:1:2)
38
39 Fimalgoritmo
  
```

Escolha uma ferramenta

Do papel aos softwares focados em fluxogramas, existem diversas opções disponíveis para montar o seu. Basicamente, qualquer plataforma que permita desenhar formas geométricas e escrever pode ser útil para criar seu modelo.

A seguir, confira algumas opções e escolha a que mais se adequa às suas necessidades:

- Word
- PowerPoint
- Microsoft Visio
- Lucid Chart
- Canva
- Asana
- Miro
- Draw.io

2.4 Exemplo do problema ao algoritmo

Problema: calcular a média de um aluno a partir de duas notas e informar se ele foi aprovado (média maior ou igual a 6).

```
Início
1. Ler a primeira nota (nota1)
2. Ler a segunda nota (nota2)
3. Calcular a média: media <- (nota1 + nota2) / 2
4. Se media >= 6, então escrever "Aprovado"
   Senão escrever "Reprovado"
Fim
```

2.5 Exercícios: Capítulo 2

1. Descreva, em passos numerados, um algoritmo para verificar se um número digitado pelo usuário é par ou ímpar.
2. Descreva um algoritmo para calcular a área de um retângulo, sabendo que o usuário informará a base e a altura.
3. Explique, com suas palavras, o que representa o símbolo de losango em um fluxograma.
4. Escreva o algoritmo (em passos numerados) para determinar o maior entre três números informados pelo usuário.

Capítulo 3: Conhecendo o VisuAlg e o Portugol

3.1 O que é o Portugol?

Portugol (contração de "Português" + "Algol") é uma pseudolinguagem de programação estruturada, escrita com comandos em português. Ela não serve para criar programas comerciais, mas é uma excelente ferramenta didática, pois permite que o aluno aprenda a lógica de programação sem a barreira de uma língua estrangeira (como o inglês, usado em linguagens como C, Java ou Python).

3.2 O que é o VisuAlg?

VisuAlg é um ambiente de desenvolvimento (editor e interpretador) gratuito, muito utilizado no ensino de lógica de programação no Brasil, que permite escrever, executar e depurar algoritmos em Portugol. Ele mostra o resultado da execução do algoritmo passo a passo, o que facilita a compreensão de como cada instrução altera os valores das variáveis.

3.3 Estrutura básica de um algoritmo em VisuAlg

Todo algoritmo escrito no VisuAlg segue uma estrutura fixa, composta por três partes principais:

```
algoritmo "Nome do Algoritmo"
// Seção de declaração de variáveis
var
    nome_variavel: tipo

inicio
    // Seção de comandos
    // (o que o algoritmo efetivamente faz)
Fimalgoritmo
```

Explicando cada parte

- **Algoritmo "...":** define o nome do programa (obrigatório, sempre entre aspas).
- **Var:** palavra reservada que inicia a seção onde declaramos todas as variáveis que serão usadas.
- **Início:** marca o começo do bloco de comandos que serão executados.
- **Fimalgoritmo:** marca o fim do programa.
- **// texto:** cria um comentário, um texto explicativo que o VisuAlg ignora ao executar o programa.

3.4 Primeiro programa: "Olá, Mundo!"

É tradição, ao aprender qualquer linguagem de programação, começar com um programa simples que exibe uma mensagem na tela. Veja como isso é feito em VisuAlg:

```
algoritmo "Ola Mundo"
inicio
    escreva("Ola, Mundo!")
fimalgoritmo
```

O comando `escreva()` é usado para exibir textos ou valores na tela. Tudo o que estiver entre aspas dentro dos parênteses será mostrado exatamente como está escrito.

Atenção

O VisuAlg pode não reconhecer corretamente acentos e caracteres especiais (ç, ã, é, ê etc.) em todas as versões e configurações. Por isso, é uma boa prática evitar acentuação dentro dos textos e comentários dos algoritmos, escrevendo, por exemplo, "Ola" em vez de "Olá".

3.5 Regras gerais de sintaxe

- O VisuAlg não diferencia maiúsculas de minúsculas (não é case sensitive) para palavras reservadas.
- Cada comando é normalmente escrito em uma linha própria.
- Nomes de variáveis não podem ter espaços, acentos ou começar com número.
- Toda string (texto) deve estar entre aspas duplas.
- O símbolo `<-` é usado para atribuir um valor a uma variável (equivale ao sinal de igual em outras linguagens).

3.6 Exercícios: Capítulo 3

1. Escreva, no VisuAlg, um algoritmo que exiba na tela seu nome completo e o curso que você está fazendo.
2. Explique com suas palavras a função das palavras reservadas `algoritmo`, `var`, `inicio` e `fimalgoritmo`.
3. Escreva um algoritmo que exiba, em linhas separadas, três frases diferentes usando o comando `escreva()`.
4. Pesquise e explique para que serve o comando `escreval()` no VisuAlg, e qual a diferença dele para o comando `escreva()`.

Capítulo 4: Variáveis, Constantes e Tipos de Dados

4.1 O que são variáveis?

Variáveis são espaços de memória utilizados para armazenar dados que podem ser alterados durante a execução do programa. Cada variável possui um nome (identificador) e um tipo de dado associado, que define que tipo de informação ela pode armazenar.

4.2 Regras para nomear variáveis

- Devem começar com uma letra (nunca com número).
- Não podem conter espaços, acentos ou caracteres especiais (use _ para separar palavras, como `media_final`).
- Não podem ser iguais a palavras reservadas da linguagem (como `inicio`, `var`, `se`, etc.).
- Devem ter nomes que representem claramente a informação armazenada (`idade`, `nome_aluno`, `salario`).

4.3 Tipos de dados primitivos no VisuAlg

Tipo de dado	Descrição	Exemplo de valor
inteiro	Números inteiros, positivos ou negativos, sem casas decimais	10, -5, 0, 200
real	Números com casas decimais (ponto flutuante)	3.14, -0.5, 10.0
caractere	Texto, cadeia de caracteres (string)	"Amazonas", "Manaus"
logico	Valor booleano: apenas verdadeiro ou falso	verdadeiro, falso

4.4 Declarando variáveis

As variáveis são declaradas na seção `var`, antes do início do bloco de comandos. Podemos declarar várias variáveis do mesmo tipo na mesma linha, separadas por vírgula:

```
algoritmo "Declaracao de Variaveis"
var
    nome: caractere
    idade: inteiro
    altura: real
    aprovado: logico
    nota1, nota2, media: real

inicio
    // comandos aqui
Fimalgoritmo
```

4.5 Constantes

Constantes são valores que não se alteram durante a execução do programa. No VisuAlg, elas são declaradas com a palavra reservada `const`, geralmente antes da seção `var`:

```

algoritmo "Uso de Constante"
const
  PI = 3.14159
var
  raio, area: real

inicio
  escreva("Digite o raio: ")
  leia(raio)
  area <- PI * raio * raio
  escreva("A area do circulo e: ", area)
fimalgoritmo

```

4.6 Atribuição de valores

A atribuição é feita com o operador <- (seta para a esquerda), que armazena um valor (ou resultado de uma expressão) dentro de uma variável:

```

idade <- 18
nome <- "Priscila"
media <- (nota1 + nota2) / 2
aprovado <- verdadeiro

```

Dica da professora

Pense na variável como uma caixa com uma etiqueta (o nome) que guarda um valor (o conteúdo). Esse conteúdo pode ser trocado a qualquer momento, mas a caixa sempre guarda apenas um valor de cada vez, do tipo definido na declaração.

4.7 Exercícios: Capítulo 4

1. Declare variáveis adequadas (com nome e tipo corretos) para armazenar: o nome de um produto, seu preço, a quantidade em estoque e se ele está em promoção.
2. Explique a diferença entre uma variável do tipo inteiro e uma do tipo real, dando um exemplo de uso para cada uma.
3. Escreva um algoritmo que declare uma constante para o valor do salário mínimo e uma variável para calcular o dobro desse valor.
4. Aponte o que está incorreto nos seguintes nomes de variáveis e reescreva-os corretamente: "2idade", "nome completo", "salário", "var".

Capítulo 5: Operadores e Expressões

5.1 Operadores aritméticos

Usados para realizar cálculos matemáticos entre valores numéricos:

Operador	Operação	Exemplo
+	Adição	$5 + 3$ resulta em 8
-	Subtração	$5 - 3$ resulta em 2
*	Multiplicação	$5 * 3$ resulta em 15
/	Divisão (sempre real)	$5 / 2$ resulta em 2.5
div	Divisão inteira	$5 \text{ div } 2$ resulta em 2
mod	Resto da divisão (módulo)	$5 \text{ mod } 2$ resulta em 1
^	Potenciação	$2 ^ 3$ resulta em 8

5.2 Operadores relacionais

Usados para comparar dois valores, sempre retornando verdadeiro ou falso:

Operador	Significado	Exemplo
=	Igual a	$5 = 5 \rightarrow$ verdadeiro
<>	Diferente de	$5 <> 3 \rightarrow$ verdadeiro
>	Maior que	$5 > 3 \rightarrow$ verdadeiro
<	Menor que	$5 < 3 \rightarrow$ falso
>=	Maior ou igual a	$5 >= 5 \rightarrow$ verdadeiro
<=	Menor ou igual a	$5 <= 3 \rightarrow$ falso

5.3 Operadores lógicos

Usados para combinar duas ou mais condições (expressões lógicas):

Operador	Significado	Exemplo
e	Verdadeiro somente se as duas condições forem verdadeiras	$(idade \geq 18) \text{ e } (possui_cnh = \text{verdadeiro})$
ou	Verdadeiro se pelo menos uma condição for verdadeira	$(chove = \text{verdadeiro}) \text{ ou } (esta_frio = \text{verdadeiro})$
nao	Inverte o valor lógico da condição	nao (aprovado)

5.4 Ordem de precedência dos operadores

Assim como na matemática, os operadores possuem uma ordem de prioridade na hora de serem calculados. Do maior para o menor grau de prioridade:

1. Parênteses ()
2. Potenciação (^)
3. Multiplicação, divisão, div e mod (*, /, div, mod)
4. Adição e subtração (+, -)
5. Operadores relacionais (=, <>, >, <, >=, <=)
6. Operador lógico NAO
7. Operador lógico E
8. Operador lógico OU

Dica da professora

Na dúvida sobre a ordem em que uma expressão será calculada, use parênteses! Eles deixam o código mais claro e evitam erros de lógica, além de ajudarem outras pessoas a entenderem seu raciocínio.

5.5 Exemplo prático

```
algoritmo "Operadores"
var
    a, b, resultado: inteiro

inicio
    a <- 10
    b <- 3
    resultado <- (a + b) * 2
    escreva("Resultado: ", resultado)
    escreva("Divisao inteira: ", a div b)
    escreva("Resto: ", a mod b)
fimalgoritmo
```

5.6 Exercícios: Capítulo 5

1. Calcule manualmente o resultado das expressões: $2 + 3 * 4$; $(2 + 3) * 4$; $10 \text{ div } 3$; $10 \text{ mod } 3$.
2. Escreva a expressão lógica em Portugol que representa: "a pessoa pode votar se tiver 16 anos ou mais".
3. Escreva um algoritmo que leia dois números inteiros e exiba a soma, a subtração, o produto, a divisão real e o resto da divisão entre eles.
4. Escreva a expressão lógica que verifica se um número é, ao mesmo tempo, maior que 0 e menor que 100.

Capítulo 6: Entrada e Saída de Dados

6.1 Comandos de saída: escreva() e escreval()

Ambos exibem dados na tela. A diferença é que `escreval()` pula uma linha após exibir o conteúdo, enquanto `escreva()` mantém o cursor na mesma linha:

```
algoritmo "Comandos de Saida"
inicio
    escreva("Isto fica ")
    escreva("na mesma linha")
    escreval("")
    escreval("Isto fica em uma linha")
    escreval("E isto em outra linha")
fimalgoritmo
```

6.2 Comando de entrada: leia()

O comando `leia()` interrompe a execução do algoritmo e aguarda que o usuário digite um valor, que será armazenado na variável indicada entre parênteses. É fundamental que a variável já tenha sido declarada com o tipo compatível com o dado que será digitado.

```
algoritmo "Entrada de Dados"
var
    nome: caractere
    idade: inteiro

inicio
    escreva("Digite seu nome: ")
    leia(nome)
    escreva("Digite sua idade: ")
    leia(idade)
    escreval("Ola, ", nome, "! Voce tem ", idade, " anos.")
fimalgoritmo
```

6.3 Concatenando textos e variáveis

É possível exibir, em um único comando `escreva()` ou `escreval()`, textos fixos e conteúdos de variáveis, separando cada elemento por vírgula:

```
escreval("O aluno ", nome, " tem
", idade, " anos e mora em
Manaus.")
```

Atenção

O comando `leia()` espera um valor compatível com o tipo da variável. Se a variável for do tipo inteiro e o usuário digitar um texto, o programa apresentará erro. Sempre planeje os tipos de dados de acordo com a informação que será realmente digitada.

6.4 Exemplo completo: Calculadora de Média

```

algoritmo "Calculadora de Media"
var
    nome: caractere
    nota1, nota2, media: real

inicio
    escreva("Digite o nome do aluno: ")
    leia(nome)
    escreva("Digite a primeira nota: ")
    leia(nota1)
    escreva("Digite a segunda nota: ")
    leia(nota2)

    media <- (nota1 + nota2) / 2

    escreval("Aluno: ", nome)
    escreval("Media final: ", media)
fimalgoritmo

```

Por que a indentação é essencial?

- Legibilidade: Facilita a leitura do código por humanos, permitindo bater o olho e entender rapidamente onde começa e onde termina um bloco de comandos.
- Manutenção: Encontrar erros (bugs) ou alterar regras em um código bem indentado é infinitamente mais rápido e seguro.
- Lógica visual: Ela reflete o raciocínio do algoritmo. Se um comando só deve ser executado mediante uma condição, ele deve estar recuado para a direita para indicar essa dependência.
- Exigência real: Em linguagens de programação modernas como Python, a indentação não é apenas estética; ela é obrigatória e dita a sintaxe. Um recuo errado fará o programa quebrar ou executar a lógica de forma incorreta.

6.5 Exercícios: Capítulo 6

1. Escreva um algoritmo que leia o nome e o preço de um produto e exiba uma mensagem informando os dois valores.
2. Escreva um algoritmo que leia dois números e exiba a soma entre eles, usando escreval().
3. Escreva um algoritmo que leia a base e a altura de um triângulo e calcule sua área (base * altura / 2).
4. Explique, com suas palavras, a diferença entre os comandos escreva() e escreval().

Capítulo 7: Estruturas Condicionais

7.1 Por que usar estruturas condicionais?

As estruturas condicionais (também chamadas de estruturas de seleção ou de decisão) permitem que o programa tome decisões, executando diferentes blocos de comando dependendo se uma condição é verdadeira ou falsa. Elas são essenciais para criar algoritmos que reagem a diferentes situações.

7.2 Seleção simples: se ... entao

```
se (condicao) entao
    // comandos executados se a condicao for verdadeira
fimse
```

Exemplo: verificar se um número é positivo.

```
algoritmo "Numero Positivo"
var
    numero: inteiro

inicio
    escreva("Digite um numero: ")
    leia(numero)

    se (numero > 0) entao
        escreval("O numero e positivo.")
    fimse
fimalgoritmo
```

7.3 Seleção composta: se ... entao ... senao

```
se (condicao) entao
    // comandos se a condicao for verdadeira
senao
    // comandos se a condicao for falsa
fimse
```

Exemplo: verificar aprovação de um aluno.

```
algoritmo "Aprovacao"
var
    media: real

inicio
    escreva("Digite a media final: ")
    leia(media)

    se (media >= 6) entao
        escreval("Aluno aprovado!")
    senao
        escreval("Aluno reprovado.")
    fimse
fimalgoritmo
```

7.4 Condições encadeadas: senao se

Quando existem mais de duas possibilidades, podemos encadear várias condições usando senao se:

```
algoritmo "Classificacao por Faixa Etaria"
var
    idade: inteiro

inicio
    escreva("Digite a idade: ")
    leia(idade)

    se (idade < 12) entao
        escreval("Crianca")
    senao se (idade < 18) entao
        escreval("Adolescente")
    senao se (idade < 60) entao
        escreval("Adulto")
    senao
        escreval("Idoso")
    fimse
fimalgoritmo
```

7.5 Seleção múltipla: escolha ... caso

Quando uma mesma variável precisa ser comparada com vários valores possíveis (por exemplo, um menu de opções), a estrutura escolha (também chamada de caso) deixa o código mais organizado do que vários se encadeados:

```
algoritmo "Menu de Opcoes"
var
    opcao: inteiro

inicio
    escreval("1 - Cadastrar")
    escreval("2 - Consultar")
    escreval("3 - Excluir")
    escreva("Escolha uma opcao: ")
    leia(opcao)

    escolha opcao
        caso 1
            escreval("Voce escolheu Cadastrar")
        caso 2
            escreval("Voce escolheu Consultar")
        caso 3
            escreval("Voce escolheu Excluir")
        outrocaso
            escreval("Opcao invalida")
    fimescolha
fimalgoritmo
```

Dica da professora

Use escolha...caso quando estiver comparando UMA mesma variável com vários valores fixos (como em um menu). Quando as condições envolverem faixas de valores ou comparações diferentes, prefira se...senao se.

7.6 Exercícios: Capítulo 7

1. Escreva um algoritmo que leia um número e informe se ele é par ou ímpar (dica: use o operador mod).
2. Escreva um algoritmo que leia três notas de um aluno, calcule a média e classifique o resultado em: "Aprovado" (média ≥ 7), "Recuperação" (média entre 5 e 6.9) ou "Reprovado" (média menor que 5).
3. Escreva um algoritmo, usando escolha...caso, que leia o número de um mês (1 a 12) e exiba o nome da estação do ano correspondente (considere o hemisfério sul).
4. Escreva um algoritmo que leia três números e exiba o maior deles, utilizando estruturas condicionais.

Capítulo 8: Estruturas de Repetição

8.1 Por que usar estruturas de repetição?

As estruturas de repetição (também chamadas de laços ou loops) permitem executar um mesmo bloco de comandos várias vezes, sem precisar reescrevê-lo. Elas são fundamentais quando não sabemos, de antemão, quantas vezes uma ação precisará ser repetida, ou quando queremos repetir uma tarefa um número exato de vezes.

8.2 Estrutura para (contador definido)

Usada quando sabemos exatamente quantas vezes o bloco de comandos deve se repetir:

```
para variavel de inicio ate fim faca
    // comandos repetidos
fimpara
```

Exemplo: exibir os números de 1 a 10.

```
algoritmo "Contagem"
var
    i: inteiro

inicio
    para i de 1 ate 10 faca
        escreval(i)
    fimpara
fimalgoritmo
```

Também é possível definir o incremento (passo) usando a cláusula passo:

```
para i de 10 ate 1 passo -1 faca
    escreval(i)
fimpara
```

8.3 Estrutura enquanto (repetição com teste no início)

O bloco de comandos é repetido enquanto a condição for verdadeira. A condição é testada antes de cada repetição por isso, se a condição já começar falsa, o bloco pode nunca ser executado.

```
enquanto (condicao) faca
    // comandos repetidos
fimenquanto
```

Exemplo: somar números digitados pelo usuário até que ele digite 0.

```
algoritmo "Soma ate Zero"
var
    numero, soma: inteiro

inicio
```

```

soma <- 0
escreva("Digite um numero (0 para parar): ")
leia(numero)

enquanto (numero <> 0) faca
    soma <- soma + numero
    escreva("Digite um numero (0 para parar): ")
    leia(numero)
fimenquanto

escreval("A soma total foi: ", soma)
fimalgoritmo

```

8.4 Estrutura repita ... ate (repetição com teste no final)

Nesta estrutura, o bloco de comandos é executado ao menos uma vez, pois a condição é verificada somente depois de cada repetição. O laço se repete até que a condição se torne verdadeira (é o oposto lógico do enquanto).

```

repita
    // comandos repetidos
ate (condicao)

```

Exemplo: pedir uma senha até que o usuário acerte.

```

algoritmo "Validar Senha"
var
    senha: inteiro

inicio
    repita
        escreva("Digite a senha (1234): ")
        leia(senha)
    ate (senha = 1234)

    escreval("Senha correta! Acesso liberado.")
fimalgoritmo

```

8.5 Comparando as três estruturas

Estrutura	Quando usar	Teste da condição
para	Número de repetições conhecido/definido	Controlado automaticamente pelo contador
enquanto	Número de repetições desconhecido; pode não executar nenhuma vez	No início do laço
repita...ate	Número de repetições desconhecido; executa ao menos uma vez	No final do laço

Atenção

Cuidado com laços infinitos! Isso acontece quando a condição de parada nunca se torna falsa (no enquanto) ou nunca se torna verdadeira (no repita...ate), geralmente porque o programador esquece de atualizar, dentro do laço, a variável usada na condição.

8.6 Exercícios: Capítulo 8

1. Escreva um algoritmo que exiba, usando a estrutura para, todos os números pares entre 1 e 50.
2. Escreva um algoritmo que calcule o fatorial de um número digitado pelo usuário (dica: fatorial de 5 é $5*4*3*2*1$).
3. Escreva um algoritmo que leia números digitados pelo usuário e pare quando for digitado um número negativo, exibindo ao final a soma e a média dos números positivos digitados.
4. Escreva um algoritmo, usando repita...ate, que simule um caixa eletrônico pedindo repetidamente uma senha de 4 dígitos até que o usuário acerte, permitindo no máximo 3 tentativas.

Capítulo 9: Vetores (Introdução)

9.1 O que é um vetor?

Um vetor (também chamado de array ou variável composta homogênea) é uma estrutura que permite armazenar vários valores do mesmo tipo em uma única variável, organizados em posições numeradas (índices). Isso evita a necessidade de criar uma variável separada para cada valor.

9.2 Declarando um vetor

```
var
  notas: vetor[1..5] de real
```

O trecho acima declara um vetor chamado notas, capaz de armazenar 5 valores reais, nas posições de 1 a 5.

9.3 Acessando e preenchendo um vetor

```
algoritmo "Vetor de Notas"
var
  notas: vetor[1..5] de real
  i: inteiro
  soma, media: real

inicio
  soma <- 0

  para i de 1 ate 5 faca
    escreva("Digite a nota ", i, ": ")
    leia(notas[i])
    soma <- soma + notas[i]
  fimpara

  media <- soma / 5
  escreval("A media da turma e: ", media)
finalgoritmo
```

Dica da professora

Cada posição de um vetor é acessada pelo seu índice, escrito entre colchetes, como em notas[1] ou notas[i]. Vetores são muito úteis quando combinados com a estrutura para, pois permitem percorrer todas as posições automaticamente.

9.4 Busca em um vetor

Uma operação muito comum é procurar (buscar) um valor dentro de um vetor, percorrendo suas posições uma a uma e comparando com o valor procurado:

```

algoritmo "Busca em Vetor"
var
    numeros: vetor[1..10] de inteiro
    i, procurado: inteiro
    encontrado: logico

inicio
    para i de 1 ate 10 faca
        escreva("Digite o numero da posicao ", i, ": ")
        leia(numeros[i])
    fimpara

    escreva("Digite o numero que deseja procurar: ")
    leia(procurado)

    encontrado <- falso
    para i de 1 ate 10 faca
        se (numeros[i] = procurado) entao
            encontrado <- verdadeiro
        fimse
    fimpara

    se (encontrado) entao
        escreval("Numero encontrado no vetor!")
    senao
        escreval("Numero nao encontrado.")
    fimse
fimalgoritmo

```

9.5 Exercícios: Capítulo 9

1. Escreva um algoritmo que leia 10 números inteiros em um vetor e exiba o maior valor digitado.
2. Escreva um algoritmo que leia um vetor de 5 nomes (caractere) e depois exiba todos eles em ordem.
3. Escreva um algoritmo que leia 5 números em um vetor e informe quantos deles são pares.
4. Escreva um algoritmo que leia 8 números em um vetor e calcule a soma e a média dos valores armazenados.

Capítulo 10: Matrizes

10.1 O que é uma matriz?

Uma matriz (também chamada de vetor bidimensional ou variável composta homogênea de duas dimensões) é uma estrutura que organiza dados em linhas e colunas, como uma tabela. Enquanto o vetor usa apenas um índice para localizar cada valor, a matriz usa dois índices: um para a linha e outro para a coluna.

10.2 Declarando uma matriz

```
var
    tabuleiro: vetor[1..3, 1..3] de inteiro
```

O trecho acima declara uma matriz chamada tabuleiro com 3 linhas e 3 colunas (uma matriz 3x3), capaz de armazenar 9 valores inteiros no total.

10.3 Acessando elementos de uma matriz

Cada elemento de uma matriz é acessado por dois índices entre colchetes, separados por vírgula: primeiro o número da linha, depois o número da coluna. Por exemplo, tabuleiro[2,3] acessa o valor da linha 2, coluna 3.

10.4 Preenchendo e exibindo uma matriz

Para percorrer todas as posições de uma matriz, usamos dois laços para aninhados (um dentro do outro): o laço externo controla a linha e o laço interno controla a coluna.

```
algoritmo "Matriz de Notas"
var
    notas: vetor[1..3, 1..2] de real
    aluno, avaliacao: inteiro

inicio
    // Preenchimento: 3 alunos, 2 notas cada
    para aluno de 1 ate 3 faca
        para avaliacao de 1 ate 2 faca
            escreva("Nota ", avaliacao, " do aluno ", aluno, ": ")
            leia(notas[aluno, avaliacao])
        fimpara
    fimpara

    // Exibicao
    para aluno de 1 ate 3 faca
        escreva("Aluno ", aluno, " -> ")
        para avaliacao de 1 ate 2 faca
            escreva(notas[aluno, avaliacao], " ")
        fimpara
        escreval("")
    fimpara
fimalgoritmo
```

Dica da professora

Pense em uma matriz como uma planilha: o primeiro índice sempre indica a linha (como se fosse a numeração das linhas de uma planilha) e o segundo índice indica a coluna. Usar nomes de variáveis como linha e coluna nos laços para deixar o código mais fácil de entender do que usar apenas i e j.

10.5 Exemplo: soma dos elementos de uma matriz

```

algoritmo "Soma da Matriz"
var
    matriz: vetor[1..2, 1..3] de inteiro
    linha, coluna, soma: inteiro

inicio
    soma <- 0

    para linha de 1 ate 2 faca
        para coluna de 1 ate 3 faca
            escreva("Digite o valor [", linha, ",", coluna, "]: ")
            leia(matriz[linha, coluna])
            soma <- soma + matriz[linha, coluna]
        fimpara
    fimpara

    escreval("A soma de todos os elementos e: ", soma)
fimalgoritmo

```

10.6 Exercícios: Capítulo 10

1. Escreva um algoritmo que leia uma matriz 3x3 de números inteiros e exiba a soma de cada linha.
2. Escreva um algoritmo que leia uma matriz 2x4 e exiba o maior valor encontrado, informando também em que linha e coluna ele está.
3. Escreva um algoritmo que leia uma matriz 3x3 e calcule a soma dos elementos da diagonal principal (posições onde linha = coluna).
4. Escreva um algoritmo que leia uma matriz 4x2 representando o nome de um produto e sua quantidade em estoque, e exiba somente os produtos com quantidade menor que 10.

Capítulo 11: Arquivos

11.1 Por que utilizar arquivos?

Até agora, todos os dados dos nossos algoritmos existiam apenas durante a execução do programa: ao fechar o VisuAlg, tudo o que foi digitado era perdido. Os arquivos permitem que um programa grave informações de forma permanente em disco, para que possam ser lidas novamente mais tarde inclusive por outro programa ou em outra execução.

11.2 Tipos de arquivo no VisuAlg

O VisuAlg trabalha com arquivos do tipo texto, ou seja, arquivos que armazenam informações em formato legível, linha a linha, sem formatação especial. Antes de usar um arquivo, é preciso declará-lo na seção var, indicando o tipo arquivo:

```
var
  arq: arquivo
```

11.3 Comandos básicos para manipular arquivos

Comando	Função
atribuir <variavel> arquivo <"caminho">	Associa a variável de arquivo a um nome/caminho de arquivo no disco
reescreva(<variavel>)	Abre o arquivo para escrita, criando-o (ou apagando o conteúdo, se já existir)
abreva(<variavel>)	Abre um arquivo já existente para leitura
escreva(<variavel>, ...)	Grava dados no arquivo aberto
leia(<variavel>, ...)	Lê dados do arquivo aberto
fecha(<variavel>)	Fecha o arquivo, salvando as alterações

11.4 Gravando dados em um arquivo

Exemplo: gravar uma lista de nomes de alunos em um arquivo de texto.

```
algoritmo "Gravar Alunos"
var
  arq: arquivo
  nome: caractere
  continuar: caractere

inicio
  atribuir arq arquivo "alunos.txt"
  reescreva(arq)

  repita
    escreva("Digite o nome do aluno: ")
```

```

        leia(nome)
        escreva(arq, nome)

        escreva("Deseja cadastrar outro aluno? (S/N): ")
        leia(continuar)
        ate (continuar = "N") ou (continuar = "n")

        fecha(arq)
        escreval("Dados gravados com sucesso!")
    fimalgoritmo

```

11.5 Lendo dados de um arquivo

Exemplo: ler e exibir todos os nomes gravados no arquivo anterior.

```

algoritmo "Ler Alunos"
var
    arq: arquivo
    nome: caractere

inicio
    atribuir arq arquivo "alunos.txt"
    abreva(arq)

    enquanto nao fimarq(arq) faca
        leia(arq, nome)
        escreval(nome)
    fimenquanto

    fecha(arq)
fimalgoritmo

```

A função `fimarq(arq)` verifica se o cursor de leitura chegou ao final do arquivo, retornando verdadeiro nesse caso. Ela é usada como condição de parada do laço `enquanto`, garantindo que todas as linhas sejam lidas, independentemente de quantas existam.

Dica da professora

Sempre feche o arquivo com `fecha()` ao terminar de usá-lo. Isso garante que os dados gravados sejam realmente salvos em disco e libera o arquivo para ser aberto por outros programas.

Atenção

Os comandos de manipulação de arquivo podem variar ligeiramente entre versões do VisuAlg (por exemplo, `abreva/reescreva` podem aparecer como `abrev/reescr` em algumas versões). Sempre consulte a documentação da versão instalada no laboratório antes de executar seus algoritmos.

11.6 Exercícios: Capítulo 11

1. Escreva um algoritmo que grave em um arquivo os nomes e as idades de 5 pessoas digitadas pelo usuário.

2. Escreva um algoritmo que leia o arquivo criado no exercício anterior e exiba na tela todas as informações gravadas.
3. Explique, com suas palavras, a diferença entre os comandos `reescreva()` e `abreva()`.
4. Escreva um algoritmo que grave em um arquivo os produtos de uma lista de compras (nome e preço) e, em seguida, leia o arquivo e exiba o valor total da lista.

Capítulo 12: Projeto Prático Final

12.1 Objetivo do projeto

Este capítulo reúne, em um único projeto, todos os conceitos estudados na apostila: variáveis, tipos de dados, operadores, entrada e saída, estruturas condicionais, estruturas de repetição, vetores, matrizes e arquivos. O objetivo é que o aluno pratique a construção de um algoritmo completo, aplicando o raciocínio lógico do início ao fim.

12.2 Proposta: Sistema de Cadastro de Produtos

Desenvolva, em VisuAlg, um algoritmo que simule um pequeno sistema de cadastro de produtos para uma loja, contendo um menu com as seguintes opções:

1. Cadastrar produto (nome e preço)
2. Listar todos os produtos cadastrados
3. Calcular o valor total do estoque (soma de todos os preços)
4. Salvar os produtos cadastrados em um arquivo de texto
5. Sair do sistema

O programa deve utilizar:

- Um vetor para armazenar os nomes dos produtos e outro para os preços.
- A estrutura escolha...caso para o menu de opções.
- A estrutura repita...ate (ou enquanto) para manter o menu em execução até que o usuário escolha sair.
- A estrutura para percorrer os vetores ao listar produtos ou calcular o total.
- Os comandos de arquivo (atribuir, reescreva, escreva, fecha) para salvar os dados em disco.

12.3 Estrutura sugerida do algoritmo

```
algoritmo "Cadastro de Produtos"
const
    MAX = 10
var
    nomes: vetor[1..MAX] de caractere
    precos: vetor[1..MAX] de real
    quantidade, opcao, i: inteiro
    total: real
    arq: arquivo

inicio
    quantidade <- 0

    repita
        escreval("===== MENU =====")
        escreval("1 - Cadastrar produto")
        escreval("2 - Listar produtos")
```

```

    escreval("3 - Valor total do estoque")
    escreval("4 - Salvar em arquivo")
    escreval("5 - Sair")
    escreva("Escolha uma opcao: ")
    leia(opcao)

    escolha opcao
    caso 1
        se (quantidade < MAX) entao
            quantidade <- quantidade + 1
            escreva("Nome do produto: ")
            leia(nomes[quantidade])
            escreva("Preco do produto: ")
            leia(precos[quantidade])
        senao
            escreval("Estoque cheio!")
        fimse

    caso 2
        para i de 1 ate quantidade faca
            escreval(nomes[i], " - R$ ", precos[i])
        fimpara

    caso 3
        total <- 0
        para i de 1 ate quantidade faca
            total <- total + precos[i]
        fimpara
        escreval("Valor total do estoque: R$ ", total)

    caso 4
        atribuir arq arquivo "produtos.txt"
        reescreva(arq)
        para i de 1 ate quantidade faca
            escreva(arq, nomes[i], ";", precos[i])
        fimpara
        fecha(arq)
        escreval("Produtos salvos em produtos.txt!")

    caso 5
        escreval("Encerrando o sistema...")

    outrocaso
        escreval("Opcao invalida!")
    fimsecolha

    ate (opcao = 5)
fimalgoritmo

```

12.4 Desafios extras

1. Adicione ao sistema uma opção para buscar um produto pelo nome e exibir seu preço.
2. Adicione uma opção para calcular e exibir o preço médio dos produtos cadastrados.
3. Modifique o sistema para não permitir o cadastro de produtos com preço negativo, exibindo uma mensagem de erro nesse caso.
4. Adicione uma opção que exiba o produto mais caro e o mais barato cadastrados.
5. Adapte o sistema para usar uma matriz de estoque por filial (por exemplo, uma matriz 3x5, representando 3 filiais com 5 produtos cada) em vez de um único vetor.

Referências

- ASCENCIO, Ana Fernanda Gomes; CAMPOS, Edilene Aparecida Veneruchi de. Fundamentos da Programação de Computadores: algoritmos, Pascal, C/C++ e Java. São Paulo: Pearson, 2012.
- FORBELLONE, André Luiz Villar; EBERSPÄCHER, Henri Frederico. Lógica de Programação: a construção de algoritmos e estruturas de dados. São Paulo: Pearson, 2005.
- MANZANO, José Augusto N. G.; OLIVEIRA, Jayr Figueiredo de. Algoritmos: lógica para desenvolvimento de programação de computadores. São Paulo: Érica, 2019.
- MEDINA, M., & FERTIG, C. Algoritmos e Programação: Teoria e Prática. São Paulo: Novatec Editora, 2005.
- PUGA, S., & RISSETTI, G.. Lógica de Programação e Estrutura de Dados com Aplicações em Java. São Paulo: Pearson Education do Brasil, 2016.
- SAID, R. Curso de Lógica de Programação. [S.l.]: Universo dos Livros, 2014.
- SOUZA, F. V. de. Lógica para a Computação. Teresina: Universidade Federal do Piauí/Universidade Aberta do Piauí, 2008.
- SOUZA, M. A. F., et al. Algoritmos e Lógica de Programação. São Paulo: Thomson Learning, 2006.
- VisuAlg: Ambiente de desenvolvimento de algoritmos em Portugal. Disponível: <https://sourceforge.net/projects/visualg30/>
- Material de apoio elaborado pela Profa. Priscila Gonçalves para o CETAM.